

Proyecto RAG: Asistente de Trámites para Familias con Miembros con Autismo en Andalucía

1. Descripción general

Las familias con hijos o hijas con Trastorno del Espectro Autista (TEA) en Andalucía se enfrentan a una gran cantidad de **trámites administrativos** (discapacidad, dependencia, ayudas económicas, escolarización, atención temprana, etc.), dispersos en diferentes portales y documentos (PDFs, BOJA, guías en web, formularios...).

El objetivo de este proyecto es desarrollar un **prototipo de sistema RAG (Retrieval-Augmented Generation)** que ayude a estas familias a **entender y navegar los trámites** más habituales, a partir de la normativa y procedimientos vigentes en Andalucía.

El sistema debe ser capaz de:

- Buscar y almacenar documentación relevante (leyes, decretos, guías, FAQs oficiales...).
- Convertir PDFs y otros formatos a texto plano.
- Limpiar la información y eliminar contenido irrelevante.
- Crear embeddings y “trocear” la información en chunks adecuados.
- Guardar esos embeddings en ficheros o en una base de datos/vector store.
- Ofrecer una función de búsqueda y generación de respuestas apoyada en esa base de conocimiento.
- Permitir realizar **pruebas y validaciones** sobre la calidad de las respuestas.

2. Alcance del proyecto

El proyecto se centrará en **trámites y recursos para familias con miembros con TEA en Andalucía**, por ejemplo (a modo orientativo):

- Reconocimiento de grado de discapacidad.
- Acceso a atención temprana (0–6 años).
- Trámites relacionados con escolarización y NEAE/NEE.
- Ayudas económicas, prestaciones y apoyos a la autonomía.

- Recursos específicos de autismo (federaciones, asociaciones, servicios especializados).
-

3. Fases y tareas principales

3.1. Búsqueda y recopilación de normativa y procedimientos

- Localizar fuentes oficiales de información (Junta de Andalucía, BOJA, portales educativos y sociales, etc.).
- Seleccionar:
 - Leyes y decretos relevantes.
 - Guías y documentos explicativos para familias.
 - Preguntas frecuentes (FAQs) y páginas de información práctica.
- Descargar la documentación (PDFs, HTML, etc.) y organizarla en una estructura de carpetas clara.

Entregable parcial:

Listado de documentos utilizados (con enlaces) y breve descripción de cada uno.

3.2. Conversión y limpieza de información (PDF → texto plano)

- Convertir los PDFs y otros formatos a **texto plano** (por ejemplo, usando librerías de extracción de texto).
- Detectar y **eliminar información inútil o ruido**, como:
 - Cabeceras y pies de página repetidos.
 - Números de página, índices automáticos que no aportan contenido.
 - Texto duplicado o contenido meramente técnico sin relación con los trámites (si procede).
- Unificar codificación y formato (UTF-8, saltos de línea coherentes, etc.).

Entregable parcial:

Carpeta con los textos limpios y, opcionalmente, un pequeño informe explicando qué tipos de “ruido” se han eliminado.

3.3. Definición de estrategia de chunking

- Diseñar una **estrategia de partición del texto en chunks**:
 - Tamaño de cada chunk (número aproximado de caracteres o tokens).
 - Posible solapamiento entre chunks para no cortar ideas importantes.
 - Criterios lógicos (por ejemplo, trocear por artículos, apartados, epígrafes).
- Implementar el proceso que:
 - Lee los textos limpios.
 - Los divide en chunks.
 - Asocia a cada chunk metadatos básicos (fuente, título del documento, sección, URL original, etc.).

Entregable parcial:

Script o módulo de chunking + ejemplo de salida (uno o varios ficheros con los chunks y sus metadatos).

3.4. Creación de embeddings y almacenamiento en vector store

- Elegir un modelo de **embeddings** (por ejemplo, un modelo de frases multilingüe o específico para español).
- Generar embeddings para cada chunk.
- Diseñar y aplicar una estrategia de almacenamiento:
 - O bien ficheros (por ejemplo, formato JSON / parquet con texto + embedding).
 - O bien una **BDD vectorial** (FAISS, Chroma, pgvector, etc.).
- Asegurar que el sistema permite:
 - Insertar nuevos documentos.
 - Recalcular y añadir nuevos embeddings si la normativa cambia.

Entregable parcial:

Base de datos/vector store con los embeddings generados y documentación breve explicando el esquema y la elección técnica.

3.5. Implementación de la función de búsqueda y RAG

- Implementar una **función de búsqueda semántica**:
 - Entrada: una pregunta en lenguaje natural (por ejemplo, “¿Qué pasos tengo que seguir para pedir la atención temprana para mi hijo?”).
 - Proceso:
 - Convertir la pregunta a embedding.
 - Buscar los chunks más relevantes en el vector store.
 - Salida: lista de chunks relevantes (texto + metadatos).
- Implementar la capa de **generación de respuesta**:
 - Usar un modelo de lenguaje que, a partir de los chunks recuperados, redacte una respuesta clara y en lenguaje comprensible para familias.
 - Incluir, en la medida de lo posible, referencias a las **fuentes** (por ejemplo, indicando la ley o guía de la que se ha extraído la información).
- Ofrecer un interfaz mínimo (CLI, API REST o pequeña web) que permita:
 - Introducir preguntas.
 - Ver las respuestas generadas.
 - Ver los fragmentos de normativa en los que se basa la respuesta.

Entregable parcial:

Función de búsqueda + módulo de respuesta RAG + interfaz mínima para consultas.

3.6. Pruebas y evaluación

- Definir un **conjunto de preguntas de prueba**, típicas de familias con hijos con TEA en Andalucía (10–20 preguntas).

- Evaluar el sistema según:
 - Pertinencia de los documentos recuperados (relevancia de los chunks).
 - Claridad y corrección de las respuestas generadas.
 - Grado de apoyo práctico para entender el trámite (por ejemplo, si explica pasos, documentos necesarios, dónde acudir, etc.).
- Documentar:
 - Errores detectados.
 - Limitaciones del sistema (por ejemplo, normativa cambiante, posibles imprecisiones, necesidad de revisión experta).
 - Propuestas de mejora futura.

Entregable parcial:

Informe de pruebas con ejemplos de preguntas, respuestas del sistema y comentarios críticos.

4. Requisitos mínimos del proyecto

- Incluir todas las fases: **búsqueda de normativa, limpieza, PDF → texto, chunking, embeddings, almacenamiento, búsqueda y pruebas.**
 - Trabajar específicamente con **normativa y recursos de Andalucía** relacionados con familias y TEA.
 - Entregar:
 1. Código fuente documentado.
 2. Conjunto de datos procesados (textos limpios y/o base vectorial).
 3. Documento de memoria del proyecto (objetivos, decisiones técnicas, resultados y evaluación).
 4. Guía breve de uso del prototipo para un usuario no técnico (familia o asociación).
-

5. Consideraciones éticas y legales

- Especificar claramente que el sistema es **de apoyo informativo y no sustituye asesoramiento jurídico ni profesional**.
- Tener en cuenta la **actualización de la normativa**: indicar la fecha de la última recopilación de documentos.
- Cuidar el lenguaje para que sea **respetuoso, accesible y comprensible** para familias con distintos niveles de formación.

7. Arquitectura backend y APIs (Node.js y/o FastAPI)

Como ampliación del proyecto, se desarrollará un **backend REST** que exponga las funciones principales del sistema RAG mediante una API.

El alumnado podrá elegir una de estas opciones (o ambas, para comparar tecnologías):

- **Opción A:** Backend con **Node.js + Express**
- **Opción B:** Backend con **Python + FastAPI**

7.1. Endpoints mínimos

Se propone, como mínimo, la implementación de los siguientes endpoints:

1. **POST /api/query**

Entrada (JSON):

```
{
  "pregunta": "¿Qué pasos debo seguir para solicitar la atención temprana para mi hijo?"
}
```

-
- **Proceso:**
 - Generar embedding de la pregunta.
 - Buscar los chunks más relevantes en el vector store.
 - Llamar al modelo de lenguaje (RAG) para generar una respuesta apoyada en esos chunks.

Salida (JSON):

```
{
  "respuesta": "Para solicitar la atención temprana en Andalucía,
debe...",
  "fuentes": [
    {
      "documento": "Ley 1/2023 de Atención Temprana",
      "url": "https://..."
    }
  ]
}
```

○

2. GET /api/health

- Endpoint de **salud del sistema**, que indique si el backend está funcionando y si puede acceder al vector store.

Respuesta simple:

```
{
  "status": "ok",
  "vector_store": "connected"
}
```

○

3. (Opcional) POST /api/reindex

- Recarga los documentos, recalcula embeddings y reconstruye el índice/vector store.
- Útil para cuando se añade o se actualiza normativa.

7.2. Backend con Node.js + Express (Opción A)

Requisitos mínimos:

- Proyecto Node.js con:
 - **express** para los endpoints.

- Una capa de servicio que:
 - Llame a la lógica de búsqueda (vector store + embeddings).
 - Orqueste la preparación del contexto para el modelo de lenguaje.
 - Organización recomendada:
 - `/src/app.js` o `/src/index.js` → servidor Express.
 - `/src/routes/` → definición de rutas (`queryRoutes`, `adminRoutes`, etc.).
 - `/src/services/` → lógica RAG (búsqueda, llamada a LLM).
 - `/src/config/` → configuración (claves API, rutas de ficheros, etc.).
-

7.3. Backend con FastAPI (Opción B)

Requisitos mínimos:

- Proyecto Python con:
 - `fastapi` para definir endpoints.
 - `uvicorn` como servidor de desarrollo/producción.
- Organización recomendada:
 - `main.py` → aplicación FastAPI.
 - `routers/` → rutas (`query.py`, `admin.py`).
 - `services/` → lógica RAG y acceso a embeddings/vector store.
 - `config/` → configuración (variables de entorno, rutas de datos).

Ejemplo de estructura de respuesta en FastAPI (que coincida con la de Node):

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()
```



```
class QueryRequest(BaseModel):
    pregunta: str

@app.post("/api/query")
def query_rag(request: QueryRequest):
    # llamar a la lógica RAG: embeddings + búsqueda + generación
    return {
        "respuesta": "Texto de respuesta generado",
        "fuentes": [
            {"documento": "Ley 1/2023", "url": "https://..."}
        ]
    }
```

(En Node.js se haría algo equivalente con Express.)

8. Despliegue del sistema

El proyecto incluirá una fase de **despliegue**, con el objetivo de que el prototipo sea accesible desde fuera del entorno de desarrollo.

10. Ampliación 2: Cliente web para familias y profesionales

Como segunda ampliación, se desarrollará un **cliente web** que permita interactuar de forma sencilla con el backend RAG (Node.js o FastAPI) y que esté pensado específicamente para:

- **Familias con miembros con TEA** (modo sencillo, lenguaje claro).
- **Profesionales / asociaciones** (modo avanzado, con más detalle y acceso a fuentes).

10.1. Objetivos del cliente web

El cliente web deberá permitir:

1. Formular preguntas en lenguaje natural sobre trámites relacionados con TEA en Andalucía
(p.ej.: “¿Cómo solicito la atención temprana?”, “¿Qué papeles necesito para el reconocimiento de discapacidad?”).

2. Mostrar la **respuesta generada por el sistema RAG** de forma clara y comprensible.
 3. Visualizar las **fuentes legales y documentos** en los que se apoya la respuesta (leyes, decretos, guías, webs oficiales, etc.).
 4. Registrar un **historial básico de consultas** durante la sesión (para que la familia pueda revisar lo que ha preguntado).
-

10.2. Requisitos funcionales mínimos

El cliente web deberá incluir, como mínimo:

1. **Página principal de consulta** con:

- Un cuadro de texto donde escribir la pregunta.
- Un botón “Consultar” que envíe la petición al endpoint `/api/query` del backend.
- Un área de resultados donde se muestre:
 - La respuesta generada.
 - Las fuentes (nombre del documento + enlace, cuando esté disponible).

2. **Historial de consultas en la sesión:**

- Mostrar una lista de las últimas preguntas realizadas y permitir volver a ver sus respuestas.

3. **Información contextual:**

- Un pequeño texto fijo que explique que el sistema:
 - Es un **apoyo informativo**.
 - No sustituye asesoramiento jurídico, médico ni administrativo profesional.
 - Está basado en normativa de Andalucía y en la fecha X (poner fecha de corte).
-

10.3. Requisitos de diseño (UX/UI)

- Interfaz **limpia y accesible**, con:
 - Texto de buen tamaño.
 - Colores neutros y buen contraste.
- Lenguaje sencillo y respetuoso, especialmente en los textos estáticos dirigidos a familias.
- Se recomienda implementar:
 - Un “**modo familia**” (vista por defecto):
 - Respuestas resumidas.
 - Mensajes en lenguaje llano.
 - Un “**modo avanzado**”:
 - Más detalle sobre la normativa concreta.
 - Lista ampliada de fuentes, artículos y enlaces.

(El cambio de modo puede ser un simple botón o un conmutador.)